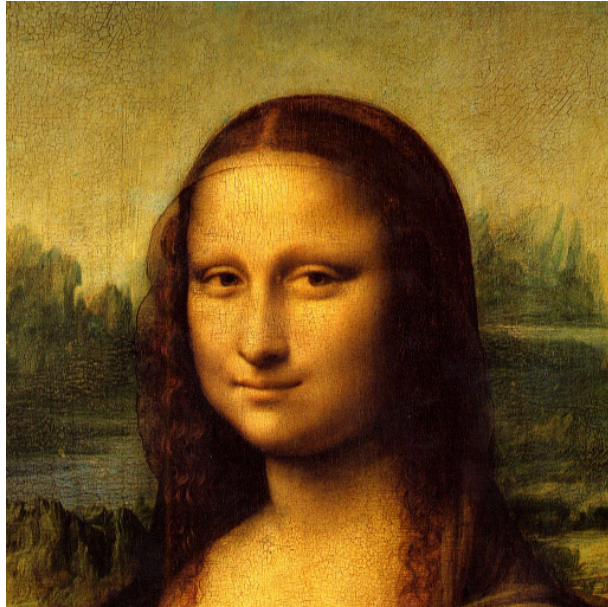




Project Proposal

Sean Larsen - seala24@student.sdu.dk

Kristóf Ákos Koltai - akkol24@student.sdu.dk

Denis Nedzhibov - dened24@student.sdu.dk

Igor Zimmermann - igzim24@student.sdu.dk

Örs Tomaj Jeney - orjen24@student.sdu.dk

Dávid Borka - dabor24@student.sdu.dk

Contents

1. Background and Motivation	3
2. Aim	3
3. Use cases	3
3.1. Booking process	3
3.2. Users interacting during consultation process	4
3.3. Review and send doctors note to patient	4
4. Initial Requirements	5
4.1. Functional Requirements	5
4.2. Non-Functional Requirements	6
5. Methods	7
5.1. Task tracking	7
5.2. Documents and Presentations	8
5.3. Repository	8
6. Architecture	8
6.1. Application Layer Diagram	8
6.1.1. Frontend Layer	9
6.1.2. Gateway Layer	9
6.1.3. Application Layer:	10
6.1.4. Infrastructure Services:	10
6.1.5. Data Layer:	10
6.2. Component-Based System Diagram	10
6.3. Tech Stack	11
7. Risks	12
7.1. Security	12
7.2. Componentized approach	12
Appendix	13

1. Background and Motivation

Outpatient departments face outdated manual processes that drastically slow down operations and put burden on both sick patients and medical staff. Doctors and nurses spend too much time on paper schedules, taking notes during consultations, and separate systems for writing prescriptions, which causes long wait times, unsatisfied patient experience (due to the disrupted doctor focus), and errors, plus burnout from extra administrative work. These issues also create serious privacy risks for patient data, which demand strict precautions and careful handling.

Our project is relevant because it offers a solution for a clear gap: the need for simple AI tools that run entirely locally using open-source models, keeping all sensitive health information secure within the clinic's network. By automating the full process; from patient booking and real-time transcription to AI-generated clinical summaries, "safety-net" suggestions, and instant digital prescriptions; it helps healthcare facilities modernize quickly and affordably. This reduces mistakes, speeds up patient flow, and relieves medical staff from the burden of endless paper work, allowing them to focus on delivering quality care and improve patient satisfaction.

2. Aim

In many outpatient departments, doctors must manually manage appointments, transcribe consultations, and write prescriptions, which disrupts the flow of patients and increases the workload of hospital staff. Our project addresses this by providing a privacy-preserving, locally hosted OPD management system that automates the workflow from consultation to digital prescription using open-source speech and large language models, with a "safety net" of suggestive alerts.

The technical aim of our project is to tackle the challenge of component-based architecture by orchestrating multiple independent services, such as appointment management, transcription, clinical summarization, medical suggestive alerts, and prescription generation. These services are isolated, each have their own runtime environment with well-defined interfaces that connect them and enable easy replacement and seamless upgrades of individual components.

The system follows a design that prioritizes privacy preservation, because dealing with sensitive health data requires strict precautions. To achieve this privacy thinking design, we will only use locally hosted open-source models, so that patient data never leaves the hospital's network.

This modular design makes it easy to replace components while preserving privacy and supporting clinical workflow.

3. Use cases

3.1. Booking process

- Primary actor (user)

- Patient
- Preconditions
 - Patient is registered in the system
 - Patient has valid login credentials
- Goal
 - Book a consultation with a doctor
- Flow
 - User navigates to client website
 - User logs in
 - User selects a preferred doctor from the availability list
 - The system displays available time slots
 - The user books an available appointment
 - The user gets redirected to a dashboard where they can:
 - See their booking(s)
 - Remove their booking(s)
 - The system sends an email as a confirmation

3.2. Users interacting during consultation process

- Primary actor (user)
 - Doctor
- Secondary actor
 - Patient
- Preconditions
 - Appointment exists in the system
 - Doctor has valid credentials
- Goal
 - Doctor and patient discuss the issues
 - Doctor identifies symptoms, concludes a diagnosis and if necessary prescribes medicine and refers to other department
 - The system internally processes the audio and transcribes the consultation
- Flow
 - User navigates to the doctor website
 - User logs in
 - The system display a dashboard with todays bookings related to the user
 - The user selects the appropriate appointment
 - The user presses start consultation
 - The system starts the audio recording
 - The system waits until the user presses end consultation
 - Audio recording is stopped and sent to process by the speech-to-text component

3.3. Review and send doctors note to patient

- Primary actor (user)
 - Doctor

- Preconditions
 - Consultation recording completed and successfully transcribed
- Goal
 - Review AI-generated documentation, validate clinical suggestions and send final doctors note
- Flow
 - The system generates a summary based on the transcription
 - The system displays the consultations dashboard:
 - Raw transcription
 - Transcription summary
 - User has the ability to either edit or accept both documents
 - If the user edits the raw transcription
 - Summary is regenerated
 - If the user edits the summary
 - Nothing is regenerated
 - When both documents are accepted the system generates suggestions based on the transcription
 - Suggestions includes:
 - Medicine
 - Referral to other departments
 - The system displays suggestions
 - User has the ability to accept or decline the suggestions
 - The system generates a doctors note
 - The doctors note is solely based on:
 - Transcription
 - Summary
 - Accepted suggestions
 - The doctors note includes:
 - Symptoms
 - Diagnosis
 - Prescriptions

4. Initial Requirements

The following requirements define the core functionality of the project. The prioritization follows the MoSCoW method.

4.1. Functional Requirements

- **MUST have**
 - The system shall allow patients to create, view and cancel appointment bookings.
 - The system shall provide secure login with role-based access for patients and doctors.
 - The system shall record and transcribe consultations locally using an open-source speech-to-text model.

- The system shall generate a structured consultation summary and prescription using a locally hosted open-source LLM.
- The system shall implement a “Suggestive Mode” that analyzes the transcript and flags potential missing clinical elements.
- Doctors shall be able to review, edit and approve or reject AI-generated summaries and suggestions.
- The system shall generate a PDF prescription after doctor approval.
- The system shall send the approved prescription to the patient via a secure local email service.
- The system shall store structured data (users, bookings) in a relational database.
- The system shall store transcripts and AI-generated content in a non-relational database.
- The system shall support user registration through secure email invitations.
- **SHOULD have**
 - The system should log AI prompts and outputs for evaluation purposes.
 - The system should allow configurable instructions for the Suggestive Mode.
 - The system should allow users to search for a doctor / see available doctors.
 - The system should allow doctors to start / end consultations.
- **COULD have**
 - The system could allow doctors to upload diagnostic images as part of a consultation.
 - The system could provide simple statistics about number of consultations and bookings.
 - The system could support switching between different local LLM models.
- **WON'T have**
 - The system will not use any cloud-based AI services.
 - The system will not include a mobile application.

4.2. Non-Functional Requirements

- **Performance**
 - The system shall transcribe consultation audio with minimal delay under normal hardware conditions.
 - The system shall generate and deliver a PDF prescription within 5 seconds after approval.
 - Booking-related requests shall respond within 3 seconds under normal usage.
- **Portability**
 - The system shall run identically on Windows, Linux and macOS.
 - All components shall be containerized for consistent deployment.
- **Scalability**
 - The system shall support at least 5 concurrent consultations without service failure.

- Each infrastructure service shall be independently replaceable without modifying other components.
- **Maintainability**
 - The project shall follow a CBSE-aligned architecture, where services and components are loosely coupled and communicate through clearly defined API interfaces.
 - Each service/component shall be replaceable or upgradable with minimal impact on the rest of the system.
 - The modular structure shall simplify debugging by isolating faults to specific services/components.
 - The defined interfaces and separation of concerns shall improve long-term maintainability and make components interchangeable.
- **Security**
 - All patient and consultation data shall remain within the local network.
 - User passwords shall be securely hashed before storage.
 - Role-based access control shall prevent unauthorized access to consultation data.
- **Reliability**
 - The system shall recover automatically from container failure.
 - No transcript or approved prescription shall be lost due to service interruption.
- **Testing Coverage**
 - Core backend logic shall include unit and integration tests.
 - At least 80% of core application logic shall be covered by automated tests.

5. Methods

Before starting out on our project, we outlined a set of rules and methods to ensure that progression through the project will be smooth and continuous. The methodology and tools used also ensure that every contribution made to the project is peer-reviewed.

5.1. Task tracking

In order to ensure that none of the tasks get lost, we make every task (development, diagramming, documenting) an issue on Jira. We complete these tasks during one-week-long Sprints, from one Monday to the next. On Mondays, we hold meetings where we both reflect on the Sprint ending that day and plan the one coming up. On Thursdays, we also hold a Stand-up meeting where everyone gives an update on the issue(s).

To ensure that each member gets tasks distributed, we give out tasks not based on quantity, but based on their actual difficulty, which we decide using Story Point Poker and use the Fibonacci numbering system.

5.2. Documents and Presentations

To create our documentation and presentations, we choose Typst. It allows us to version control and handle our documents, as if they were code. Templates have also been made to create a uniform design and look to all of our reports and presentations. In order to keep track of our images and diagrams, we also decided to store them in the same repository as our documents and code. We also create a meeting log document during each meeting to keep everyone accountable and for anyone to catch up if they were absent.

5.3. Repository

To make sure code contribution is safe, each issue has its own branch, and pull requests must be opened to merge. Those pull requests must then be reviewed by at least two non-contributing peers. In order to have time for review and changes before the end of the sprint, pull requests must be opened at least two days before the end of the sprint. To help the work of the reviewers and to guarantee a smooth workflow, a Pull Request template has also been made, see Appendix Figure C. We merge all pull requests, during our Monday meetings, so we can resolve possible merge conflicts together. We also created a pipeline that sends a message to our Discord about a new pull request, and replying to that message, the PR owner tags the requested reviewers. This is done because not everyone checks their emails, which is GitHub's default notification channel.

6. Architecture

This semester project will be structured using a layered architecture. Implementing a component-based system (CBS) framework focusing on modularity, replaceability & reusability. Components communicate via an API Gateway implementing a service orchestration pattern.

6.1. Application Layer Diagram

This section analyses the Application diagram, see Figure 1. Breaking down the individual layers of this layered architecture.

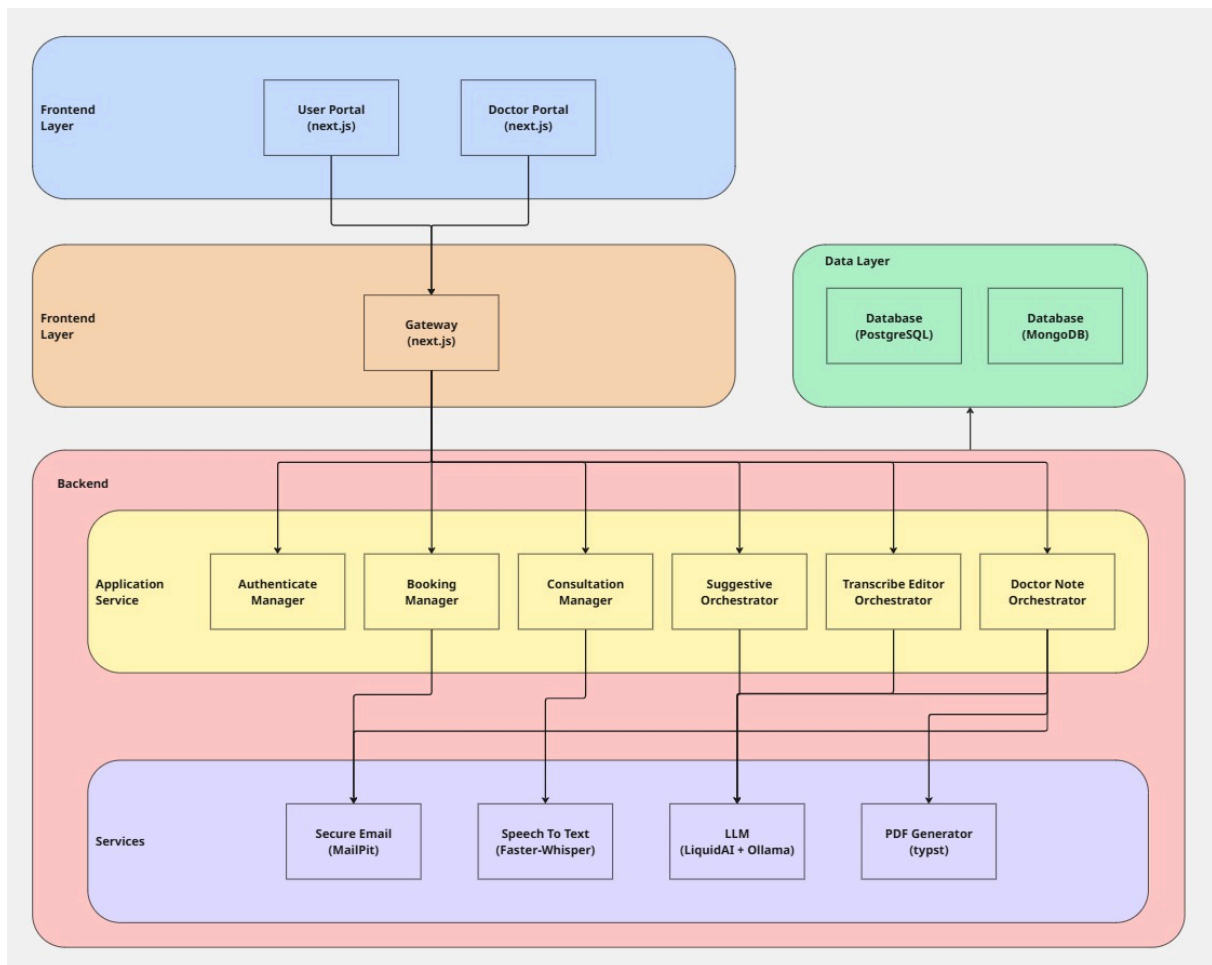


Figure 1: Application Layer Diagram

6.1.1. Frontend Layer

The main responsibility of this layer is to act as an intermediate between the user and the backend services, by triggering API calls to the orchestration logic.

- **User Portal (Next.js/Typescript):** Provides user interface for booking management and authentication for clients. Allows users to search for doctors, see availability and create/remove bookings
- **Doctor Portal (Next.js/Typescript):** Provides doctor interface for selecting bookings and starting consultation. Allows doctor to record consultation, edit transcription, accept/deny LLM generated suggestions and send prescription notes.

6.1.2. Gateway Layer

The main responsibility of this layer is to route frontend requests to the corresponding backend services (e.g. logging in, booking creation, start consultation). This provides a centralised role-based authentication to protect backend endpoints.

- **Gateway (Next.js):** Provides an interface for backend access for the frontend.

6.1.3. Application Layer:

The main responsibility of this layer is the implementation of orchestration logic to coordinate workflows between services and databases.

- **Authentication Manager:** Uses relational database to authenticate login requests
- **Booking Orchestrator:** Uses relational database to read/write data whilst creating / removing bookings. Additionally uses the 'Secure Email' service to send booking confirmation to clients.
- **Consultation Manager:** Uses 'Speech To Text' service to transcribe consultation audio to text and write it into non-relational database.
- **Suggestive Orchestrator:** Reads consultation data from the non-relational database and forwards it to the 'LLM' service to create suggestions. Stores accepted LLM generated suggestions.
- **Transcribe Editor Orchestrator:** Allows doctor to correct any transcription errors and rewrites them using 'LLM' service.
- **Doctor Note Orchestrator:** Uses doctor approved suggestions and consultation to create a doctor note using 'PDF Generator', 'LLM' and 'Secure Email' service.

6.1.4. Infrastructure Services:

The infrastructure services exposed through stable HTTP interfaces provides specialised functions. These services are isolated Docker containers exposed via an HTTP interface for modularity and follow CBS framework.

- **Speech-to-Text (Faster-Whisper):** Transcribes audio files
- **LLM Runtime (Ollama + model):** Ollama acts as an intermediary between HTTP requests and the LLM. We have yet to decide on a specific model/s.
- **PDF Generation (Typst):** Produces exportable PDF files.
- **Secure Email (Mailpit):** Sends generated outputs to corresponding clients

6.1.5. Data Layer:

The main responsibility of this layer is data persistence.

- **Relational Database (PostgreSQL):** Stores structured data such as users, doctors and booking information.
- **Non-relational Database (MongoDB):** Stores unstructured data such as transcripts, accepted AI suggestions, AI summaries and generated doctor notes.

6.2. Component-Based System Diagram

The Component-Based System (CBS) see Figure 2, illustrates how the system is structured into modular components with exposed interfaces. It can be broken down into three main layers: user interface, orchestrators/managers and services & databases. This design allows individual components to be independently developed and interchanged at runtime.

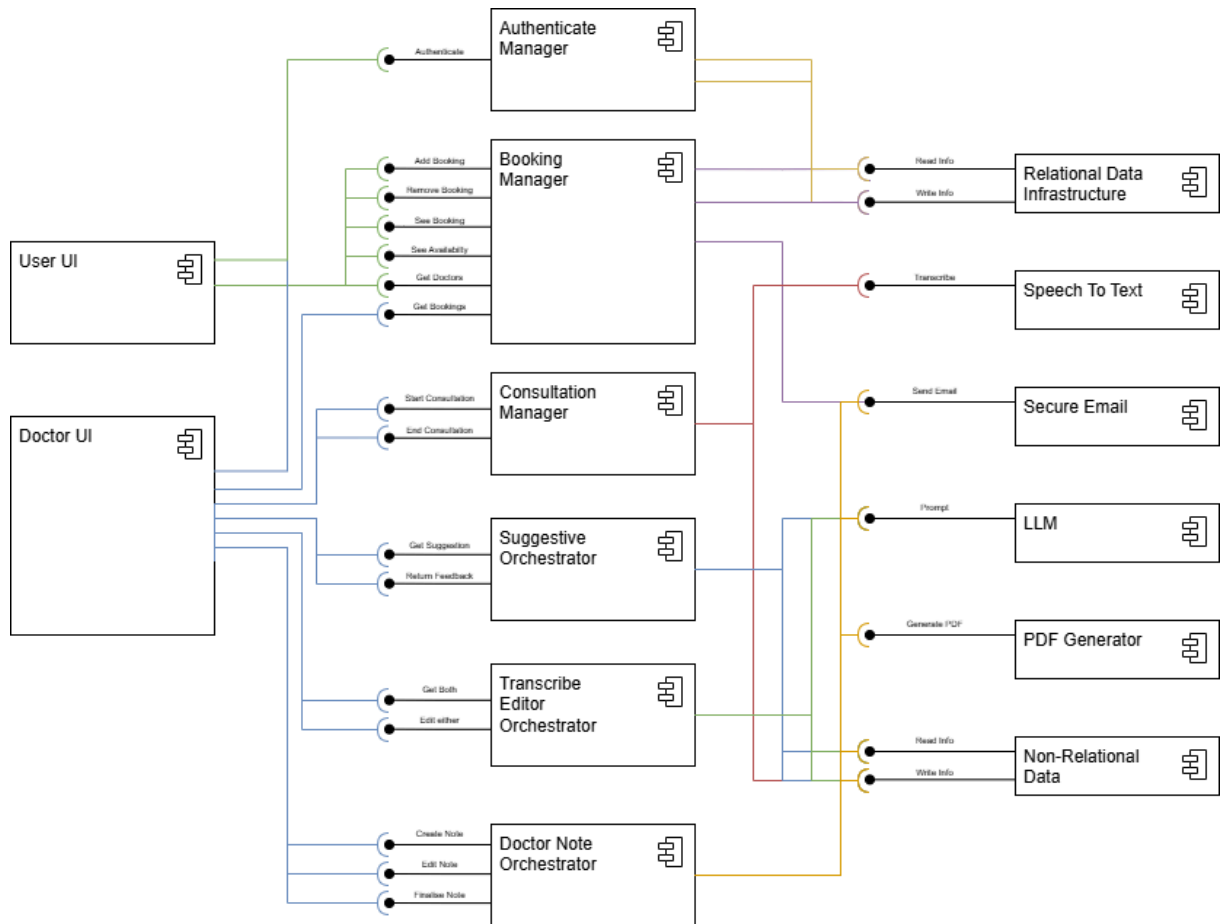


Figure 2: Component-Based System Diagram

6.3. Tech Stack

Layer	Technology	Purpose
Frontend	Next.js / Typescript	User/Doctor interface
Gateway	Next.js	API entry point & routing
Application Services	FastAPI	Orchestration & business logic
Speech-to-Text	Faster-Whisper	Audio transcription
LLM Runtime	Ollama + LLM models	Local AI
PDF Generator	Typst	PDF generation
Email	Mailpit	Email sending
Data	PostgreSQL + MongoDB	Un/structured storage
Deployment	Docker Compose	Containerised services & networking
CI (OPTIONAL)	GitHub Actions	Automated test & build checks

Table 1: Application tech stack

7. Risks

In this section we will identify potential risk and security factors. As this project will be used in professional medical environments, making sure that sensitive personal data is handled correctly is our top priority.

7.1. Security

To ensure this, we will be implementing two separate databases. Our PostgreSQL database will be used to store user data such as login, while our MongoDB database will be storing documents generated during the doctor-patient discussions, and doctor notes.

This approach keeps sensitive data separate, even if one of the database is compromised, the other one will still be secure.

Our system will be locally hosted including the LLM, which means no data will be sent to external servers, further reducing the risk of data breaches.

Our API gateway will implement JWT token authentication, so unauthorized access attempts will be rejected, before they can reach the services. Authentication and authorization will be handled by the Authentication Manager component which will be responsible to ensure that users can only access appropriate data based on their permissions.

7.2. Componentized approach

The componentized architecture addresses the risk of tight coupling. By breaking down the whole system into smaller components and separating responsibilities, we can ensure that if one fails, others can still function. This approach also allows us to easily find issues and replace components without affecting the whole system.

To achieve this we will be using Docker containerization, to isolate each component. This allows us to scale components as needed, and run health checks and automatic recovery. This would mean we could periodically check the availability of components and automatically restart them if they fail to respond, reducing downtime and improving reliability.

Appendix

<KEY> <Name of issue>

(eg. OPUS-0 Write a poem)

Description

Give a description of the issue and how you resolved it.

Important changes

.

Checklist

- ☐ My contribution follows the style guidelines of this project
- ☐ I have performed a self-review of my own contribution
- ☐ I have made corresponding changes in documentation
- ☐ My changes generate no new warnings or errors
- ☐ I have moved my task to `In Review` on Jira

Appendix Figure C: Pull Request template